Código, explicação e refatorização

Proposta 10 — Regularização de sinistros. À esquerda o código atual, ao centro o que cada bloco faz e à direita sugestões de melhoria (a verde, código já testado: a versão refatorizada completa dá o mesmo CSV e o mesmo resumo). As melhorias são opcionais: o código atual já cumpre o enunciado.

Código atual	O que faz	Como melhorar / refatorizar
1. funcoes_auxiliares.py — constantes e normalizar_ramo		

`MAPEAMENTO_RAMO = {`
`  "Automovel": "Automovel",`
`  "Automóvel": "Automovel",`
`  "Habitacao": "Habitacao",`
`  "Habitação": "Habitacao",`
`  "Viagem": "Viagem",`
`  "Responsabilidade Civil": "ResponsabilidadeCivil",`
`  "ResponsabilidadeCivil": "ResponsabilidadeCivil",`
`}`
`LIMITES = {"Automovel": 25, "Habitacao": 25,`
`  "Viagem": 10, "ResponsabilidadeCivil": 35}`
 	
`def normalizar_ramo(ramo):`
`  if pd.isna(ramo):`
`  return ramo`
`  valor = str(ramo).strip()`
`  return MAPEAMENTO_RAMO.get(valor, valor)`
	O dicionário traduz variantes de escrita do ramo para o nome padrão; `LIMITES` guarda os dias máximos por ramo.
	
	`normalizar_ramo` devolve o valor tal como está se for vazio; senão remove espaços e procura o nome padrão. O `.get(valor, valor)` evita erro quando o ramo não existe no dicionário.
		Avisar de ramos desconhecidos. Hoje um ramo novo passa em silêncio e `cumpre_prazo` assume que cumpre o prazo, o que pode esconder atrasos.
		
		Menos entradas no dicionário: comparar sem acentos nem maiúsculas, em vez de listar cada variante.
		
		Docstrings curtas nas funções (o script original tinha-as).
		
		`import unicodedata`
		`def _chave(txt):`
		`  txt = unicodedata.normalize("NFKD", str(txt))`
		`  sem_acentos = txt.encode("ascii", "ignore").decode()`
		`  return sem_acentos.replace(" ", "").lower()`
		
		`MAPEAMENTO_RAMO = {_chave(r): r for r in LIMITES}`
		
		`def normalizar_ramo(ramo):`
		`  if pd.isna(ramo):`
		`  return ramo`
		`  return MAPEAMENTO_RAMO.get(_chave(ramo), ramo)`

2. funcoes_auxiliares.py — limite_regularizacao e cumpre_prazo		

`def limite_regularizacao(ramo):`
`  return LIMITES.get(normalizar_ramo(ramo))`
 	
`def cumpre_prazo(ramo, dias_resolucao):`
`  limite = limite_regularizacao(ramo)`
`  if limite is None or pd.isna(dias_resolucao):`
`  return True`
`  return dias_resolucao <= limite`
	`limite_regularizacao`: normaliza o ramo e devolve o limite em dias (ou `None` se o ramo for desconhecido).
	
	`cumpre_prazo`: sem limite ou sem dias, devolve `True` (não assinala atraso sem informação); caso contrário compara com <=, logo o limite é inclusivo.
		A lógica está correta e clara. Melhorias possíveis:
		
		Documentar o pressuposto (sem informação = dentro do prazo) numa docstring, porque é uma decisão de negócio que o avaliador vai querer ver.
		
		Testes rápidos com `assert` nos casos-limite (25 dias em Habitacao, 10 em Viagem, ramo desconhecido, dias em falta).
		
		Alternativa vetorizada (para tabelas grandes; o enunciado pede usar as funções, por isso mantém o `apply` no trabalho):
		
		`assert cumpre_prazo("Habitacao", 25) is True`
		`assert cumpre_prazo("Viagem", 11) is False`
		`assert cumpre_prazo("Viagem", float("nan")) is True`
		
		`# vetorizado (testado: mesmo resultado)`
		`lim = df["ramo"].map(LIMITES)`
		`df["fora_prazo"] = (lim.notna()`
		`  & df["dias_resolucao"].notna()`
		`  & (df["dias_resolucao"] > lim))`

3. Ficheiro principal — imports, constantes e classe Sinistro

```
from funcoes_auxiliares import normalizar_ramo, ...
from pathlib import Path
import pandas as pd

COL_ID = "sinistro"
COL_RAMO = "ramo"
...

class Sinistro:
    def __init__(self, id_sinistro, ramo,
                 estado, valor_reclamado):
        self.id_sinistro = id_sinistro
        self.ramo = ramo
        self.estado = estado
        self.valor_reclamado = valor_reclamado

    def valor_pendente(self):
        return (self.valor_reclamado
                if self.estado == "Pendente" else 0)
```

Importa as funções e as bibliotecas e define constantes com os nomes das colunas (evita repetir texto e gralhas).

A classe guarda os dados de um sinistro e o método `valor_pendente()` aplica a mesma regra que a coluna calculada: valor reclamado se Pendente, 0 caso contrário.

Ordem dos imports (convenção PEP 8): biblioteca padrão, terceiros, módulos próprios.

`@dataclass` elimina o `__init__` repetitivo e dá um `__repr__` útil para depurar.

Constantes de estado: definir `ESTADO_PENDENTE = "Pendente"` em vez de repetir o texto em vários sítios.

Verifica que a classe é **usada com um pendente** (a demonstração com `iloc[0]` dá 0).

```
from dataclasses import dataclass

ESTADO_PENDENTE = "Pendente"

@dataclass
class Sinistro:
    id_sinistro: str
    ramo: str
    estado: str
    valor_reclamado: float

    def valor_pendente(self):
        if self.estado == ESTADO_PENDENTE:
            return self.valor_reclamado
        return 0
```

4. main() — passo 1: leitura e qualidade dos dados

```
base_dir = Path(__file__).resolve().parent
csv_file = base_dir / "dados_trabalho_10_seguros.csv"
if not csv_file.exists():
    raise FileNotFoundError(...)

df = pd.read_csv(csv_file)
df[COL_RAMO] = df[COL_RAMO].map(normalizar_ramo)
df[COL_DIAS] = pd.to_numeric(df[COL_DIAS], errors="coerce")
df[COL_RECLAMADO] = pd.to_numeric(
    df[COL_RECLAMADO], errors="coerce").fillna(0)
df[COL_REGULARIZADO] = pd.to_numeric(
    df[COL_REGULARIZADO], errors="coerce").fillna(0)

print(df.dtypes)
print(df.isna().sum())
print("Duplicados:", df.duplicated().sum())
print(df[COL_ESTADO].unique(), df[COL_RAMO].unique())
```

Localiza o CSV na pasta do script e dá erro claro se não existir. Lê com `pd.read_csv`, normaliza o ramo e converte as colunas numéricas (o que não for número passa a vazio).

Os `print` são a verificação de qualidade: tipos, valores em falta, duplicados e categorias existentes.

Separar em funções (`carregar_dados`, `verificar_qualidade`): o `main` fica curto e cada parte é testável.

Validar as colunas obrigatórias logo após ler o CSV, com mensagem clara.

Cuidado com `fillna(0)` no valor reclamado: transforma um dado em falta em 0 e esconde o problema. Melhor registar quantos valores faltam (a verificação de qualidade já os mostra, mas só *depois* de preenchidos, por isso nunca aparecem).

Ordem: faz a verificação de qualidade antes de limpar.

```
def carregar_dados(caminho):
    if not caminho.exists():
        raise FileNotFoundError(caminho)
    df = pd.read_csv(caminho)
    em_falta = set(COLUNAS_OBRIGATORIAS) - set(df.columns)
    if em_falta:
        raise ValueError(f"Colunas em falta: {sorted(em_falta)}")
    df[COL_RAMO] = df[COL_RAMO].map(normalizar_ramo)
    df[COL_ESTADO] = df[COL_ESTADO].astype(str).str.strip()
    for col in (COL_DIAS, COL_RECLAMADO, COL_REGULARIZADO):
        df[col] = pd.to_numeric(df[col], errors="coerce")
    return df
```

5. main() — passos 2 e 3: valor_pendente e fora_prazo

```
df["valor_pendente"] = df[COL_RECLAMADO].where(
    df[COL_ESTADO].astype(str).str.strip().eq("Pendente"), 0
)

df["fora_prazo"] = ~df.apply(
    lambda linha: cumpre_prazo(linha[COL_RAMO], linha[COL_DIAS]),
    axis=1,
)
```

`where(condição, 0)` mantém o valor reclamado nas linhas Pendente e põe 0 nas outras.

`apply(..., axis=1)` aplica `cumpre_prazo` a cada linha; o ~ inverte o resultado (fora de prazo = não cumpre).

Repetição: a expressão `astype(str).str.strip().eq("Pendente")` aparece também no filtro do passo 6. Calcula-a uma vez numa coluna booleana `pendente` e reutiliza-a.

Limpar o estado uma só vez ao carregar os dados (já feito no `carregar_dados` acima).

Evitar alterar o DataFrame original dentro de funções (`df.copy()`).

```
def adicionar_colunas(df):
    df = df.copy()
    pendente = df[COL_ESTADO].eq("Pendente")
    df["pendente"] = pendente
    df["valor_pendente"] = (df[COL_RECLAMADO]
                            .where(pendente, 0).fillna(0))
    df["fora_prazo"] = ~df.apply(
        lambda l: cumpre_prazo(l[COL_RAMO], l[COL_DIAS]), axis=1)
    return df
```

6. main() — passo 4: resumo por ramo (groupby)

```
resumo = df.groupby(COL_RAMO).agg(
    n_sinistros=(COL_ID, "count"),
    valor_reclamado=(COL_RECLAMADO, "sum"),
    valor_regularizado=(COL_REGULARIZADO, "sum"),
    valor_pendente=("valor_pendente", "sum"),
    dias_medios=(COL_DIAS, "mean"),
    pct_fora_prazo=("fora_prazo",
                    lambda s: s.mean() * 100),
).round(2)
print(resumo)
```

Agrupa por ramo e calcula, por ramo: número de sinistros, somas dos valores, média de dias e percentagem fora de prazo (a média de verdadeiros/falsos dá a proporção; ×100 converte em %).

Evitar a `lambda`: usar `"mean"` diretamente (mais legível e mais rápido) e multiplicar por 100 depois.

Colocar o resumo numa função que devolve a tabela, em vez de a imprimir; assim pode ser testada e reutilizada.

Total geral: acrescentar uma linha de total ajuda na leitura do relatório.

Nota analítica: nos sinistros pendentes, `dias_resolucao` é provavelmente tempo decorrido, por isso a média de dias deve ser lida com cuidado.

```
def resumo_por_ramo(df):
    resumo = df.groupby(COL_RAMO).agg(
        n_sinistros=(COL_ID, "count"),
        valor_reclamado=(COL_RECLAMADO, "sum"),
        valor_regularizado=(COL_REGULARIZADO, "sum"),
        valor_pendente=("valor_pendente", "sum"),
        dias_medios=(COL_DIAS, "mean"),
        pct_fora_prazo=("fora_prazo", "mean"),
    )
    resumo["pct_fora_prazo"] *= 100
    return resumo.round(2)
```

7. main() — passo 5: demonstração da classe

```
linha = df.iloc[0]
sinistro = Sinistro(
    linha[COL_ID], linha[COL_RAMO],
    linha[COL_ESTADO], linha[COL_RECLAMADO]
)
print(sinistro.id_sinistro,
      sinistro.valor_pendente())
```

Cria uma instância de `Sinistro` com os dados da primeira linha do CSV e chama o método. Cumpre o requisito de usar a classe no programa.

A primeira linha é o S001 (Fechado), logo o resultado é 0 e a demonstração não mostra o método a fazer nada. Usa o primeiro sinistro pendente (S002, 6 200).

O relatório em PDF menciona S002 com 6 200: este ajuste torna script e relatório coerentes.

```
primeiro = df[df["pendente"]].iloc[0]
sinistro = Sinistro(primeiro[COL_ID], primeiro[COL_RAMO],
                    primeiro[COL_ESTADO],
                    primeiro[COL_RECLAMADO])
print(sinistro.id_sinistro, sinistro.valor_pendente())
# S002 6200
```

8. main() — passo 6: exportação dos prioritários

```
prioritarios = df[
    (df[COL_ESTADO].astype(str).str.strip().eq("Pendente"))
    | df["fora_prazo"]
]
prioritarios = prioritarios.sort_values(
    by=["valor_pendente", COL_DIAS],
    ascending=[False, False]
)
prioritarios.to_csv(
    base_dir / "sinistros_prioritarios.csv", index=False)
```

Seleciona os sinistros pendentes **ou** fora de prazo (`|` é o OU entre colunas booleanas), ordena por valor pendente e dias de resolução, ambos decrescentes, e grava em CSV sem índice.

Reutilizar a coluna `pendente` em vez de repetir a expressão do passo 2.

Desempate estável: acrescentar o id do sinistro como terceiro critério garante sempre a mesma ordem em caso de empate (hoje os sinistros fechados têm todos valor pendente 0).

Função própria `selecionar_prioritarios` para a regra de seleção; a gravação fica no `main`.

Não exportar colunas auxiliares (a coluna `pendente`) com `drop(columns=...)`.

```
def selecionar_prioritarios(df):
    prio = df[df["pendente"] | df["fora_prazo"]]
    return prio.sort_values(
        by=["valor_pendente", COL_DIAS, COL_ID],
        ascending=[False, False, True], kind="stable")

prio = selecionar_prioritarios(df)
prio.drop(columns="pendente").to_csv(
    base_dir / "sinistros_prioritarios.csv", index=False)
```

9. Estrutura geral do programa

```
def main():
    # 1. leitura e qualidade
    # 2. valor_pendente
    # 3. fora_prazo
    # 4. resumo por ramo
    # 5. classe
    # 6. exportação
    # apoio à análise (prints)

if __name__ == "__main__":
    main()
```

Um único `main()` faz tudo, por passos comentados. O bloco `if __name__ == "__main__"` só executa o programa quando o ficheiro é corrido diretamente.

Divisão em funções pequenas, cada uma com uma responsabilidade; o `main` passa a ler-se como o índice do trabalho:

Prints de apoio: os prints finais por canal servem a análise, mas no relatório convém apresentar tabelas, não prints soltos.

Resultado: a versão completa (página seguinte) foi corrida e dá exatamente o mesmo CSV e o mesmo resumo que o código atual.

```
def main():
    base_dir = Path(__file__).resolve().parent
    df = carregar_dados(base_dir / "dados_trabalho_10_seguros.csv")
    verificar_qualidade(df)
    df = adicionar_colunas(df)
    print(resumo_por_ramo(df))
    # demonstração da classe ...
    prio = selecionar_prioritarios(df)
    prio.drop(columns="pendente").to_csv(
        base_dir / "sinistros_prioritarios.csv", index=False)
```

Anexo — versão refatorizada completa (testada)

Corre com `funcoes_auxiliares.py` e o CSV de dados na mesma pasta. Se a usares no trabalho, confirma que percebes cada função: a avaliação inclui a explicação do código na apresentação.

```
"""Proposta 10 - Regularização de sinistros (versão refatorizada)."""
from dataclasses import dataclass
from pathlib import Path

import pandas as pd

from funcoes_auxiliares import cumpre_prazo, normalizar_ramo

COL_ID, COL_RAMO, COL_CANAL = "sinistro", "ramo", "canal"
COL_ESTADO, COL_DIAS = "estado", "dias_resolucao"
COL_RECLAMADO, COL_REGULARIZADO = "valor_reclamado", "valor_regularizado"
COLUNAS_OBRIGATORIAS = [COL_ID, COL_RAMO, COL_CANAL, COL_ESTADO, COL_DIAS,
                        COL_RECLAMADO, COL_REGULARIZADO]


@dataclass
class Sinistro:
    """Representa um sinistro e calcula o respetivo valor pendente."""
    id_sinistro: str
    ramo: str
    estado: str
    valor_reclamado: float

    def valor_pendente(self):
        return self.valor_reclamado if self.estado == "Pendente" else 0


def carregar_dados(caminho):
    if not caminho.exists():
        raise FileNotFoundError(f"Ficheiro de dados não encontrado: {caminho}")
    df = pd.read_csv(caminho)
    em_falta = set(COLUNAS_OBRIGATORIAS) - set(df.columns)
    if em_falta:
        raise ValueError(f"Colunas em falta no CSV: {sorted(em_falta)}")
    df[COL_RAMO] = df[COL_RAMO].map(normalizar_ramo)
    df[COL_ESTADO] = df[COL_ESTADO].astype(str).str.strip()
    for col in (COL_DIAS, COL_RECLAMADO, COL_REGULARIZADO):
        df[col] = pd.to_numeric(df[col], errors="coerce")
    return df


def verificar_qualidade(df):
    print(df.dtypes)
    print(df.isna().sum())
    print("Duplicados:", df.duplicated().sum())
    print(df[COL_ESTADO].unique(), df[COL_RAMO].unique())


def adicionar_colunas(df):
    df = df.copy()
    pendente = df[COL_ESTADO].eq("Pendente")
    df["valor_pendente"] = df[COL_RECLAMADO].where(pendente, 0).fillna(0)
    df["fora_prazo"] = ~df.apply(
        lambda l: cumpre_prazo(l[COL_RAMO], l[COL_DIAS]), axis=1)
    df["pendente"] = pendente
    return df


def resumo_por_ramo(df):
    resumo = df.groupby(COL_RAMO).agg(
```

```
        n_sinistros=(COL_ID, "count"),
        valor_reclamado=(COL_RECLAMADO, "sum"),
        valor_regularizado=(COL_REGULARIZADO, "sum"),
        valor_pendente=("valor_pendente", "sum"),
        dias_medios=(COL_DIAS, "mean"),
        pct_fora_prazo=("fora_prazo", "mean"),
    )
    resumo["pct_fora_prazo"] *= 100
    return resumo.round(2)


def selecionar_prioritarios(df):
    prioritarios = df[df["pendente"] | df["fora_prazo"]]
    return prioritarios.sort_values(
        by=["valor_pendente", COL_DIAS, COL_ID],
        ascending=[False, False, True], kind="stable")


def main():
    pd.set_option("display.width", 200)
    pd.set_option("display.max_columns", None)
    base_dir = Path(__file__).resolve().parent

    df = carregar_dados(base_dir / "dados_trabalho_10_seguros.csv")
    verificar_qualidade(df)
    df = adicionar_colunas(df)
    print(resumo_por_ramo(df))

    primeiro = df[df["pendente"]].iloc[0]
    sinistro = Sinistro(primeiro[COL_ID], primeiro[COL_RAMO],
                        primeiro[COL_ESTADO], primeiro[COL_RECLAMADO])
    print(sinistro.id_sinistro, sinistro.valor_pendente())

    prioritarios = selecionar_prioritarios(df)
    prioritarios.drop(columns="pendente").to_csv(
        base_dir / "sinistros_prioritarios.csv", index=False)


if __name__ == "__main__":
    main()
```